SBA301 – SLOT 23

CHAPTER 15 – IMPLEMENT CRUD APPLICATION WITH RESTFUL WEB SERVICE AND REACTJS

PART B: REACT CRUD CLIENT • AXIOS • FORMS • UI STATE • API INTEGRATION

CORE: Tư duy trung tâm của Slot 23: Frontend CRUD không chỉ là "gọi API rồi render". Sinh viên phải hiểu state là nguồn sự thật của UI, Effect là điểm đồng bộ với external system, Axios là boundary HTTP, form là state machine nhỏ, và mỗi thao tác Create/Read/Update/Delete phải có lifecycle rõ: user action → state → request → server response → state transition → UI feedback → verification.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	23
Chủ đề	Chapter 15 Part B – React CRUD Client, Axios, Forms
CLO chính	CLO4 – Apply full-stack SPA integration and CRUD client workflow
Ngữ liệu xuyên suốt	Orchid Management (course baseline) → FUNewsManagement transfer
Môi trường khóa học	Node.js LTS • npm • Vite • React • React Router • Axios • React-Bootstrap (nếu dùng) • Spring Boot API ở localhost:8080
Thời lượng gợi ý	135 phút trên lớp + bài tập tự luyện/assignment ngoài giờ
Learning flow	Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer
Scope lock	Tập trung frontend CRUD client. Không dạy lại sâu JPA/Repository/Controller của Slot 22; backend được xem là API contract đã có. Security/JWT để các slot sau.

CÁCH SỬ DỤNG GIÁO TRÌNH

Slot 23 tiếp nối Slot 22: backend CRUD đã tồn tại, frontend phải tiêu thụ contract đó một cách có cấu trúc. Tài liệu giữ đúng sequence của Chapter 15 Part B: Vite setup → App.jsx → NavBar.jsx → ListOfOrchids.jsx → Create modal/form → EditOrchid.jsx → run React client; đồng thời mở rộng lifecycle, debugging và production-direction để sinh viên hiểu bản chất thay vì sao chép code.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Kiểm tra Node/npm, backend API, .env và baseline React project.	Version check + GET smoke test + Git checkpoint
Understand	Phân biệt component/state/effect/form/API client; hiểu CRUD UI lifecycle.	State/API contract map
Trace	Trace GET và POST từ click/render qua Axios tới backend rồi quay về state/UI.	Execution trace + Network evidence
Implement	Hoàn thiện list/create/edit/delete với forms và routing/modal.	Source + browser evidence
Break	Cố ý sai API URL, field name, dependency array, validation hoặc CORS.	Failure evidence
Diagnose	Đọc Console, Network, React warning, Axios error object.	Debug note
Verify	Kiểm chứng UI, HTTP status, response body và DB hậu điều kiện.	Test matrix + API/DB evidence
Transfer	Thiết kế CRUD client cho News/Category/Account trong Assignment 02.	Frontend design worksheet

NOTE: AI rule: AI hỗ trợ khám phá workflow, refactor và cập nhật ecosystem; sinh viên phải đối chiếu source code, browser DevTools, Network tab, backend logs và official documentation. Không dùng AI response thay cho evidence chạy thật.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 20: SPA architecture, React ↔ Spring Boot, API URL, origin/CORS, Vite environment variables.
- Slot 22: REST CRUD backend, HTTP methods/status codes, JSON request/response, validation/error contract.
- React căn bản: component, JSX, props, useState, event handler, conditional rendering, list rendering và key.
- JavaScript: object/array, spread syntax, destructuring, async/await, Promise, try/catch, map/filter.
- HTTP: GET/POST/PUT/DELETE; headers; Content-Type; request body; status 2xx/4xx/5xx.
- Browser DevTools: Console, Network, Request/Response headers, payload, status, timing.

0.2. Công cụ và baseline

Thành phần	Yêu cầu	Kiểm tra nhanh
Node.js	LTS phù hợp môi trường môn học	node -v
npm	Đi kèm Node	npm -v
VS Code	Có ES7+ React snippets/formatter tùy chọn	code .
Backend API	Spring Boot CRUD chạy ở localhost:8080	GET /orchids/ → 200
Vite project	React + JavaScript	npm run dev
Axios	HTTP client	npm ls axios
React Router	Navigation nếu dùng	npm ls react-router-dom
React-Bootstrap	UI/modal nếu course project dùng	npm ls react-bootstrap bootstrap
Browser DevTools	Chrome/Edge	F12 → Network/Console
Git	Baseline sạch	git status

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
useState giữ dữ liệu UI; Axios gửi HTTP; backend trả JSON.	useEffect chạy khi nào? Form controlled khác uncontrolled ra sao?	Sau khi POST thành công, UI nên cập nhật bằng refetch hay local state?

0.4. Pre-flight checklist

- Backend GET /orchids/ trả 200 và JSON hợp lệ.
- CORS cho phép origin React dev server nếu frontend/backend khác origin.
- Không hard-code database password hoặc secret vào frontend.
- Có file .env chứa VITE_API_URL và biết rằng VITE_* sẽ được bundle cho client.
- Commit/tag gợi ý: s23-before-react-crud.
- Network tab đang bật Preserve log trong workshop trace.

```
node -v
npm -v
curl http://localhost:8080/orchids/
# hoặc PowerShell:
# Invoke-RestMethod http://localhost:8080/orchids/
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Slot 22 → Slot 23: từ API contract đến interactive client

Nền tảng Slot 22	Đã có	Câu hỏi Slot 23 trả lời
REST endpoints	GET/POST/PUT/DELETE + status/body	Component nào gọi endpoint nào?
JSON contract	Field names và validation rule	Form state map vào JSON ra sao?
Error contract	400/404/409/500	UI hiển thị lỗi và phục hồi thế nào?
Database persistence	Backend ghi/đọc dữ liệu thật	Sau mutation, UI đồng bộ lại bằng chiến lược nào?
CORS/API URL	FE gọi được BE	Cấu hình API client sao cho dễ đổi môi trường?

1.2. Mục tiêu học tập

1. Giải thích lifecycle của một React CRUD client từ render đầu tiên đến HTTP response và re-render.

2. Tạo React project bằng Vite và cấu hình environment variable cho API base URL.
3. Tổ chức API access qua Axios instance/service thay vì rải URL trong component.
4. Dùng useState để quản lý collection, form state, loading, error và modal/edit state.
5. Dùng useEffect đúng mục đích để đồng bộ component với external system (backend API).
6. Implement READ list/detail; CREATE; UPDATE; DELETE với Axios và UI feedback.
7. Xây dựng controlled form, parse/convert type và validation phía client mà không thay backend validation.
8. Dùng React Router hoặc modal flow phù hợp cho edit/create.
9. Debug Axios/React bằng Console + Network; phân biệt lỗi client, CORS, network, HTTP và server.
10. Kiểm chứng end-to-end bằng UI + HTTP + backend/DB hậu điều kiện.
11. Nhận biết production extensions: request cancellation, interceptors, server-state libraries, accessibility, optimistic UI, error boundaries và test automation.

1.3. Mental model của Slot 23

```
Initial render
  -> component function executes
  -> state initialized
  -> JSX committed to DOM
  -> useEffect runs after commit
  -> Axios GET /orchids/
  -> browser HTTP stack / CORS
  -> Spring Boot API
  <- JSON response
  -> setOrchids(data)
  -> React schedules re-render
  -> UI displays rows/cards

User submits form
  -> onSubmit(event)
  -> preventDefault()
  -> validate/normalize form state
  -> Axios POST/PUT
  <- 201/200 or 4xx/5xx
  -> update/refetch client state
  -> close/reset form
  -> render success/error feedback
```

1.4. Mức độ hoàn thành

Mức	Biểu hiện	Rủi ro/ý nghĩa
Demo beginner	Component gọi axios trực tiếp; hard-code URL; alert lỗi.	Chạy được nhưng khó bảo trì/test.
Course-ready	API client riêng; state/loading/error rõ; controlled form; đúng CRUD contract; DevTools verify.	Đạt mục tiêu Slot 23.
Production-ready direction	Typed contract, server-state strategy, cancellation, accessibility, standardized notifications, tests, observability.	Cần design/tooling ngoài core lab.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. React component là hàm render từ state/props

Không xem component như “màn hình tĩnh”. Mỗi render tạo snapshot của props/state; event handler và Effect phải được hiểu trong snapshot đó.

2.2. State design cho CRUD client

Tách collection state, selected/editing item, form state, loading, error và UI state. Tránh một object khổng lồ chứa mọi thứ nếu lifecycle khác nhau.

2.3. useEffect cho external synchronization

Effect phù hợp cho fetch/sync với API sau commit; không dùng Effect để tính dữ liệu thuần có thể derive trong render.

2.4. Axios boundary

Axios instance gom baseURL, headers, timeout và interceptors tùy chọn. Component gọi use case/API function thay vì lặp URL.

2.5. Controlled forms

value được điều khiển bởi state; onChange cập nhật state. Checkbox dùng checked; number/select cần chú ý conversion.

2.6. CRUD lifecycle

READ tạo state từ server; CREATE/UPDATE/DELETE là mutation và phải quyết định refetch hay patch local state.

2.7. Loading/error/success states

Không chỉ happy path. UI cần phân biệt initial loading, mutation loading, empty, validation error, HTTP error và success feedback.

2.8. API contract fidelity

Tên field, type, URL, method, status và error shape phải bám backend contract; frontend không tự "đoán".

2.9. Routing/modal design

List/detail/edit có thể dùng route; create/edit có thể dùng modal. Chọn theo URL semantics, deep-link và complexity.

2.10. Verification

Network tab là bằng chứng request thật; DB/API verify là bằng chứng mutation thật. UI đúng không đủ nếu backend state sai.

2.11. CRUD ↔ React ↔ HTTP mapping

UI use case	React event/state	Axios	HTTP	Expected
Load list	useEffect → setItems	api.get("/")	GET	200 + array
Create	onSubmit(form)	api.post("/", payload)	POST	201 + created
Edit	load selected → submit	api.put(`/${id}`, payload)	PUT	200 + updated
Delete	confirm → handler	api.delete(`/${id}`)	DELETE	204/200 theo contract
Detail	route/modal selected id	api.get(`/${id}`)	GET	200/404

2.12. Những hiểu sai cần loại bỏ

- "useEffect là nơi viết mọi logic" → sai; Effect dành cho synchronization với external system.
- "State update xảy ra ngay lập tức" → sai; React schedule update và render lại.
- "Axios lỗi nào cũng nằm ở error.response" → sai; có thể là network/setup error.
- "Client validation thay backend validation" → sai; client validation cải thiện UX, backend vẫn là authority.
- "POST xong chỉ cần push local item là luôn đúng" → không luôn đúng; server có thể sinh ID/default/normalized fields.
- "VITE_API_URL là secret" → sai; VITE_* được expose vào client bundle.
- "Không thấy lỗi Console nghĩa là request thành công" → sai; phải xem Network/status/body.
- "DELETE 204 phải có JSON" → sai; 204 thường không có body.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Case study xuyên suốt – Orchid Management

Use case	UI	Backend contract	Concern
Browse orchids	List/cards/table	GET /orchids/	loading, empty, refresh
Create orchid	Modal/form	POST /orchids/	validation, 201, reset
Edit orchid	Edit route/form	GET + PUT /orchids/{id}	prefill, dirty fields, 404
Delete orchid	Confirm dialog	DELETE /orchids/{id}	confirmation, 204, stale UI
View detail	Modal/route	GET /orchids/{id}	deep-link or selected state

3.2. Transfer sang FUNewsManagementSystem

Module	Frontend concern	Form complexity	Backend/business concern
Category	CRUD + parent/status	select parent, active flag	không xóa nếu đang được NewsArticle dùng
NewsArticle	CRUD + tags/category	long text, multi-select tags	relations, createdBy/updatedBy, status
SystemAccount	CRUD/search	role select, email validation	delete constraint, future auth/security
Profile	view/update own data	partial editable fields	authorization ở later slot

3.3. Beginner design vs production direction

Vấn đề	Beginner	Production direction
API URL	Hard-code trong component	env + API client/config
Request code	axios gọi rải rác	service/api module hoặc generated client
Server state	useState + refetch thủ công	server-state library khi complexity đủ lớn
Errors	alert(error)	standardized UI + error mapping
Form	một handler đơn giản	schema/rule rõ; form library khi form lớn
Mutation	refetch tất cả	invalidate/query cache hoặc targeted update
Concurrency	last write wins	version/ETag/optimistic locking awareness
UX	spinner toàn trang	granular pending state, disable action, retry
Testing	manual only	component/integration/E2E phù hợp
Observability	console.log	client error telemetry/correlation khi production

IV. NỘI DUNG CHI TIẾT – NỀN TẢNG LÝ THUYẾT VÀ HỆ SINH THÁI KIẾN THỨC

Phần IV là knowledge base của Slot 23. Mục tiêu là giúp sinh viên hiểu đầy đủ nền tảng trước khi bước sang workbook thực hành ở Phần VI. Các mục dưới đây được trình bày theo hierarchy khái niệm → lifecycle → semantics → trade-off → liên hệ hệ sinh thái.

4.1. React render model và component lifecycle

4.1.1. Render là gì?

Component function được gọi để tính JSX từ props/state. Render phải giữ tính thuần tương đối; side effect không nên chạy trực tiếp trong thân render.

4.1.2. Commit phase

Sau render, React cập nhật DOM. Effect chạy sau commit để đồng bộ với external system.

4.1.3. Re-render triggers

State update, parent render/props change hoặc context change có thể làm component render lại.

4.1.4. Snapshot semantics

Event handler nhìn thấy state của render đã tạo ra handler đó. Đây là nền tảng để hiểu stale closure và async behavior.

4.2. State modeling cho CRUD client

4.2.1. Domain/server data

Danh sách Orchid và selected Orchid đến từ backend.

4.2.2. Form state

Draft mà người dùng đang chỉnh; chưa chắc đã tồn tại trên server.

4.2.3. UI state

Modal open, selected tab, confirm dialog, toast.

4.2.4. Async state

loading, submitting, deletingId, error.

4.2.5. Derived state
Filtered count hoặc display label nên tính từ state gốc khi có thể thay vì duplicate state.

4.3. useEffect và external synchronization

4.3.1. Khi nên dùng
Fetch data, subscription, browser API hoặc third-party widget.

4.3.2. Dependency array
Dependencies phải phản ánh reactive values được dùng trong Effect.

4.3.3. Cleanup
Hủy subscription/request hoặc bỏ effect cũ khi dependency đổi/unmount.

4.3.4. Race condition
Hai request có thể trả về không theo thứ tự; cần cancellation/ignore strategy khi scenario phức tạp.

4.3.5. Strict Mode development
Effect có thể được stress-test trong development; code phải chịu được setup/cleanup hợp lý.

4.4. Axios và HTTP client architecture

4.4.1. Axios instance
Tạo instance với baseURL và common headers để giảm duplication.

4.4.2. Request config
method, url, params, data, headers, timeout, signal.

4.4.3. Response model
status, data, headers; application thường quan tâm data + status.

4.4.4. Error taxonomy
HTTP error có response; network error có request nhưng không có response; setup error xảy ra trước khi gửi.

4.4.5. Interceptors
Hữu ích cho token/header/logging/error normalization nhưng dễ tạo global side effect nếu lạm dụng.

4.4.6. Cancellation
AbortController/signal giúp tránh update state từ request không còn cần thiết.

4.5. REST semantics từ góc nhìn frontend

4.5.1. GET
Safe/read operation; cache semantics tùy hệ thống.

4.5.2. POST
Create/command; thường trả 201 + representation hoặc Location.

4.5.3. PUT
Update/replacement theo contract; thường idempotent.

4.5.4. PATCH
Partial update; cần contract rõ hơn; không mặc định thay PUT.

4.5.5. DELETE
Mutation có hậu điều kiện; 204 không có body là bình thường.

4.5.6. Status-driven UI
200/201/204 là success variants; 400/404/409/500 cần UX khác nhau.

4.6. Forms trong React

4.6.1. Controlled input
value/checked đến từ state; onChange cập nhật state.

4.6.2. Uncontrolled input
DOM giữ value; có thể dùng ref/FormData trong scenario đơn giản.

4.6.3. Input type semantics
text, textarea, checkbox, number, select có event/value semantics khác nhau.

4.6.4. Type conversion
HTML input values thường là string; boolean/number cần normalize trước khi gửi API.

4.6.5. Form submit
preventDefault, validate, build payload, mutation, feedback, reset/close.

4.6.6. Client validation
Required/length/pattern/business hint giúp UX nhưng không thay validation backend.

4.7. CRUD state transition patterns

4.7.1. Refetch after mutation
Đơn giản, server là source of truth; đổi lại thêm request.

4.7.2. Patch local state
Nhanh và ít request; phải merge đúng response server.

4.7.3. Optimistic update
UI đổi trước response; cần rollback khi lỗi; chỉ nên dùng khi semantics phù hợp.

4.7.4. Pessimistic update
Chờ server success rồi đổi UI; dễ hiểu và an toàn hơn cho người mới.

4.8. Error handling và resilient UX

4.8.1. Validation errors
Hiển thị gần field; giữ user input.

4.8.2. Not found/stale item
Thông báo item đã bị xóa/thay đổi; refresh list.

4.8.3. Conflict
Thông báo business conflict; không chỉ “Something went wrong”.

4.8.4. Network/offline
Retry có kiểm soát; không đánh đồng với 500.

4.8.5. Global vs local errors
Error boundary cho render errors; API errors thường được handle trong async workflow.

4.9. Routing và navigation

4.9.1. Route-based edit
URL /orchids/:id/edit hỗ trợ deep-link, refresh, history.

4.9.2. Modal-based edit
Nhanh cho admin CRUD; URL semantics có thể kém nếu không sync route.

4.9.3. Route params
id là string từ URL; convert/validate trước khi dùng khi cần.

4.9.4. Navigation after mutation
Sau create/update có thể navigate hoặc close modal tùy workflow.

4.10. API configuration và environment

4.10.1. Vite env
VITE_* được expose vào client và được thay thế khi build.

4.10.2. Không chứa secret
API base URL không phải secret; password/token private key không được đưa vào VITE_*.

4.10.3. Mode files
.env, .env.local, .env.development, .env.production hỗ trợ môi trường khác nhau.

4.10.4. Dev proxy
Có thể proxy /api trong dev để giảm hard-coded origin/CORS friction; production thường dùng reverse proxy/gateway.

4.11. Component decomposition

4.11.1. Page/container
Orchestrate fetching/navigation/state ở mức page khi phù hợp.

4.11.2. Presentational component
Nhận props và render; giảm coupling với API.

4.11.3. API/service module
Chứa HTTP calls và contract mapping.

4.11.4. Reusable form
Create/Edit có thể dùng chung OrchidForm với initialValues/mode.

4.11.5. Custom Hook awareness
Khi logic fetch/form lặp nhiều, custom hook có thể encapsulate, nhưng không cần ép dùng ở lab nhỏ.

4.12. Server-state ecosystem

4.12.1. useState/useEffect baseline
Phù hợp để học workflow nền tảng và app nhỏ.

4.12.2. Server-state libraries
TanStack Query/SWR-like tools giải quyết cache, dedupe, refetch, invalidation khi app phức tạp.

4.12.3. Không nhảy tool quá sớm
Sinh viên phải hiểu HTTP/state/effect trước khi dùng abstraction cao hơn.

4.13. Performance awareness

4.13.1. Không tối ưu sớm
Đo trước khi memoize.

4.13.2. Large lists
Pagination/filter từ server thay vì tải toàn bộ.

4.13.3. Debounce search
Giảm request khi search-as-you-type; vẫn cần cancel/stale strategy.

4.13.4. Image/network
Ảnh CDN lớn ảnh hưởng UX; dùng kích thước hợp lý/lazy loading khi production.

4.14. Accessibility và form UX

4.14.1. Label association
Mỗi input cần label rõ; placeholder không thay label.

4.14.2. Error announcement
Thông báo validation/error cần đủ ngữ nghĩa; focus management quan trọng với modal.

4.14.3. Disable pending action
Tránh double submit; vẫn cần server-side idempotency/constraint khi nghiệp vụ quan trọng.

4.15. Testing boundaries

4.15.1. Unit logic
Pure formatter/validator dễ unit test.

4.15.2. Component test
Render + user interaction + mocked network.

4.15.3. Integration/E2E
Frontend thật gọi backend/test environment để verify contract.

4.15.4. Mock Service Worker awareness
Mock network ở browser/node boundary thay vì mock axios internals có thể giữ test gần thực tế hơn.

4.16. Security awareness (chuẩn bị cho slot sau)

4.16.1. XSS/data rendering
React escape text mặc định; cẩn trọng dangerouslySetInnerHTML.

4.16.2. Token storage
Không quyết định token strategy chỉ vì “dễ”; cần cân nhắc XSS/CSRF và backend security model.

4.16.3. Authorization
Ẩn nút trên UI không phải authorization; backend vẫn phải enforce.

4.16.4. CORS
CORS không phải authentication/authorization.

4.17. Hai execution lifecycle phải trace được

4.17.1. READ lifecycle

```
React render
 -> useEffect
 -> orchidApi.getAll()
 -> axiosInstance.get("/")
 -> browser sends GET
 -> Spring Boot returns 200 + JSON
 -> Axios resolves response
 -> setOrchids(response.data)
 -> React re-render
 -> list rows/cards updated
```

4.17.2. CREATE lifecycle

```
User fills controlled form
 -> onChange updates form state
 -> onSubmit prevents native submit
 -> client-side validation/normalization
 -> orchidApi.create(payload)
 -> Axios POST JSON
 -> backend validates/persists
 <- 201 + created representation
 -> merge/refetch server state
 -> reset/close form
 -> success feedback
 -> list re-render
```

V. KINH NGHIỆM THỰC TIỄN, TIPS

Vấn đề	Triệu chứng	Cách xử lý
Sai API base URL	Network ERR_CONNECTION_REFUSED/404	Kiểm tra .env, port, path; restart Vite sau đổi .env
CORS	Browser chặn dù Postman chạy	Kiểm tra Origin, preflight, backend CORS config
useEffect loop	Request chạy liên tục	Rà dependency và state update bên trong Effect
Stale UI sau mutation	DB đúng nhưng list chưa đổi	Refetch hoặc update local state từ server response
Field name mismatch	Form gửi nhưng backend null/400	So JSON payload với contract/backend DTO
Checkbox sai	true/false không đúng	Dùng checked và e.target.checked
Number thành string	Backend type error	Number(value) / parse phù hợp trước POST/PUT
Delete double-click	Gửi 2 request	Disable action theo deletingId/pending
Axios error mơ hồ	Chỉ log message	Inspect error.response/status/data, error.request
Secret trong VITE_*	Lộ trong bundle	Chỉ để public runtime config; secret ở backend
Hard-code request trong nhiều component	Khó đổi API/test	Centralize api/service module
Form create/edit copy-paste	Bug lệch validation	Dùng reusable form khi complexity tăng

5.1. Debugging decision tree

```
UI action fails
 |- No request in Network?
 |   -> event handler / preventDefault / conditional / disabled button
 |- Request exists but blocked?
 |   -> CORS / mixed content / browser policy
 |- 404?
 |   -> baseURL/path/id mismatch
 |- 400?
 |   -> payload shape/type/validation
 |- 409?
 |   -> business conflict / stale state
 |- 500?
 |   -> backend log / server bug
 |- 2xx but UI wrong?
     -> response.data mapping -> setState -> render condition -> key/state merge
```

```
Request repeats unexpectedly
 -> inspect useEffect dependencies -> state update inside Effect -> StrictMode awareness
```

5.2. Production-minded tips

- Giữ API boundary nhỏ và rõ: component không nên biết mọi chi tiết Axios config.
- Dùng server response sau POST/PUT làm source để nhận generated ID/default/normalized fields.
- Không disable CORS hoặc dùng wildcard bừa để "chạy được".
- Không lưu secret trong frontend env; browser code luôn có thể bị xem.
- Dùng granular pending state để UX không khóa toàn trang nếu chỉ đang delete một row.
- Có confirm cho destructive action; production có thể cần undo/soft-delete tùy nghiệp vụ.
- Test negative cases từ sớm: backend không chạy, 404, validation 400, delete conflict.
- Khi app lớn, cân nhắc server-state/form libraries dựa trên pain point thật, không theo trend.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG, CỦNG CỐ KIẾN THỨC

Case study xuyên suốt: FUNewsManagementSystem
Backend baseline: Slot 22 – Chapter 15 Part A
Frontend: ReactJS + Vite + Axios + React Router + React Bootstrap
Module thực hành: Category Management
Backend API: `http://localhost:8080/api/categories`
Phần VI là **implementation workbook**. Mỗi bước phải bảo đảm:
Mục tiêu → Thao tác → Code/cấu hình → Lệnh chạy → Kiểm chứng → PASS/FAIL evidence
Frontend trong Slot 23 **không xây API giả/mock**. Sinh viên phải kết nối trực tiếp với **Category REST API đã hoàn thành ở Slot 22**.

6.0. Contract baseline trước khi code frontend

Trước khi tạo React project, xác minh backend Slot 22 đang hoạt động.

API contract

API	Method	Expected
`/api/categories`	GET	`200` + Spring `Page<CategoryResponse>`
`/api/categories/{id}`	GET	`200` + Category hoặc `404`
`/api/categories`	POST	`201` + Category vừa tạo
`/api/categories/{id}`	PUT	`200` + Category sau cập nhật
`/api/categories/{id}`	DELETE	`204` nếu xóa thành công
`/api/categories?keyword=academic&page=0&size=10`	GET	`200` + filtered Page
POST invalid payload	POST	`400`
POST duplicate category name	POST	`409`
DELETE Category đang được NewsArticle sử dụng	DELETE	`409`

CategoryResponse baseline

```
{
  "categoryId": 1,
  "categoryName": "Academic news",
  "categoryDescription": "Academic announcements",
  "parentCategoryId": null,
  "active": true
}
```

CategoryRequest baseline

```
{
  "categoryName": "Technology News",
  "categoryDescription": "Technology and software engineering news",
  "parentCategoryId": null,
  "active": true
}
```

Kiểm tra backend bằng browser/Postman/curl

```
curl "http://localhost:8080/api/categories?page=0&size=10&sort=categoryName,asc"
```

Hoặc mở:

```
http://localhost:8080/api/categories?page=0&size=10
```

PASS

- Backend chạy port `8080`.
- HTTP status = `200`.
- JSON có field `content`.
- `content` chứa Category seed từ SQL Server.

FAIL
Không debug React nếu backend chưa PASS.

6.1. Bước 1 – Kiểm tra Node/npm và tạo React project bằng Vite

Mục tiêu

Tạo React SPA đóng vai trò client cho FUNewsManagementSystem.

Thao tác

```
node -v
npm -v

cd D:\LearningCode

npm create vite@latest funews-react-client
```

Chọn:

```
Framework: React
Variant: JavaScript
```

Tiếp tục:

```
cd funews-react-client
npm install
code .
```

Chạy project:

```
npm run dev
```

Vite thường cung cấp:

```
http://localhost:5173
```

PASS

- `package.json` tồn tại.
- `node_modules` được tạo.
- Trang Vite React hiển thị.
- Browser Console không có lỗi nền tảng.

6.2. Bước 2 – Cài dependencies cho CRUD Client

```
npm install axios react-router-dom
npm install react-bootstrap bootstrap
```

Kiểm tra:

```
npm ls axios react-router-dom react-bootstrap bootstrap
```

Vai trò

Dependency	Mục đích
`axios`	Gọi Spring Boot REST API
`react-router-dom`	Điều hướng List/Edit
`react-bootstrap`	UI components
`bootstrap`	CSS baseline

React-Bootstrap là lựa chọn UI của bài thực hành. Sinh viên có thể thay bằng CSS/UI library khác nếu vẫn bảo đảm đầy đủ chức năng và contract.

6.3. Bước 3 – Tạo cấu trúc frontend

```
src/
├─ api/
│  ├─ axiosClient.js
│  └─ categoryApi.js
│
├─ components/
│  ├─ AppNavBar.jsx
│  ├─ CategoryForm.jsx
│  └─ PaginationBar.jsx
│
├─ pages/
│  ├─ CategoryList.jsx
│  └─ EditCategory.jsx
│
├─ App.jsx
├─ main.jsx
└─ index.css
.env
```

Quy tắc trách nhiệm

`api/`

Chỉ xử lý HTTP communication.

`components/`

Các UI component có khả năng tái sử dụng.

`pages/`

Quản lý page-level state và use case UI.

`App.jsx`

Chỉ tổ chức routing/layout.

Không đặt toàn bộ CRUD trong `App.jsx`.

6.4. Bước 4 – Cấu hình `.env`

Tạo file:

```
.env
```

Nội dung:

```
VITE_API_URL=http://localhost:8080/api
```

Kiểm tra:
```
console.log(import.meta.env.VITE_API_URL);
```
Sau khi thay `.env`, phải restart:
```
npm run dev
```
Security rule
`VITE_*` được bundle xuống browser.
Không đặt vào `.env` frontend:
```
SQL Server password
JWT signing secret
private key
database credential
server secret
```
6.5. Bước 5 – Tạo Axios Client
Tạo:
```
src/api/axiosClient.js
```
```
import axios from "axios";

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: {
    "Content-Type": "application/json",
  },
  timeout: 10000,
});

export default axiosClient;
```
Mục tiêu
Các component/page không hard-code:
```
http://localhost:8080
```
nhiều lần.
6.6. Bước 6 – Tạo Category API Module
Tạo:
```
src/api/categoryApi.js
```
```
import api from "./axiosClient";

export function getCategories({
  keyword = "",
  page = 0,
  size = 10,
  sort = "categoryName,asc",
} = {}) {
  return api.get("/categories", {
    params: {
      keyword: keyword || undefined,
      page,
      size,
      sort,
    },
  });
}

export function getCategoryById(id) {
  return api.get(`/categories/${id}`);
}

export function createCategory(payload) {
  return api.post("/categories", payload);
}

export function updateCategory(id, payload) {
  return api.put(`/categories/${id}`, payload);
}

export function deleteCategory(id) {
  return api.delete(`/categories/${id}`);
}
```
Kiến thức cần chú ý
Backend Slot 22 trả về:
```
Page<CategoryResponse>
```
do đó:
```
response.data
```
không phải array trực tiếp.
Danh sách thật nằm tại:
```
response.data.content
```
Metadata phân trang nằm tại:
```
response.data.number
response.data.size
response.data.totalElements
response.data.totalPages
response.data.first
response.data.last
```

Đây là khác biệt quan trọng so với bài Orchid cũ.

6.7. Bước 7 – Import Bootstrap và cấu hình `main.jsx`

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";

import "bootstrap/dist/css/bootstrap.min.css";
import "./index.css";

import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

6.8. Bước 8 – Tạo Routing trong `App.jsx`

```
import { Navigate, Route, Routes } from "react-router-dom";

import AppNavBar from "./components/AppNavBar";
import CategoryList from "./pages/CategoryList";
import EditCategory from "./pages/EditCategory";

export default function App() {
  return (
    <>
      <AppNavBar />

      <Routes>
        <Route
          path="/"
          element={<Navigate to="/categories" replace />}
        />

        <Route
          path="/categories"
          element={<CategoryList />}
        />

        <Route
          path="/categories/:id/edit"
          element={<EditCategory />}
        />

        <Route
          path="*"
          element={<Navigate to="/categories" replace />}
        />
      </Routes>
    </>
  );
}
```

Checkpoint

Mở:

`http://localhost:5173/categories`

PASS: React Router render `CategoryList`.

6.9. Bước 9 – Tạo Navigation Bar

Tạo:

`src/components/AppNavBar.jsx`

```
import {
  Container,
  Nav,
  Navbar,
} from "react-bootstrap";

import { Link } from "react-router-dom";

export default function AppNavBar() {
  return (
    <Navbar
      bg="dark"
      data-bs-theme="dark"
      className="mb-4"
    >
      <Container>
        <Navbar.Brand
          as={Link}
          to="/categories"
        >
          FUNews Management
        </Navbar.Brand>
```

```
        <Nav>
          <Nav.Link
            as={Link}
            to="/categories"
          >
            Categories
          </Nav.Link>
        </Nav>
      </Container>
    </Navbar>
  );
}
```

6.10. Bước 10 – READ: Category List + Pagination + Search

Tạo:

```
src/pages/CategoryList.jsx
import {
  useCallback,
  useEffect,
  useState,
} from "react";

import {
  Alert,
  Badge,
  Button,
  Col,
  Container,
  Form,
  Row,
  Spinner,
  Table,
} from "react-bootstrap";

import { Link } from "react-router-dom";

import {
  deleteCategory,
  getCategories,
} from "../api/categoryApi";

import CategoryForm from "../components/CategoryForm";
import PaginationBar from "../components/PaginationBar";

export default function CategoryList() {
  const [categories, setCategories] = useState([]);

  const [page, setPage] = useState(0);
  const [size] = useState(10);

  const [totalPages, setTotalPages] = useState(0);
  const [totalElements, setTotalElements] = useState(0);

  const [keywordInput, setKeywordInput] = useState("");
  const [keyword, setKeyword] = useState("");

  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const [showCreate, setShowCreate] = useState(false);
  const [deletingId, setDeletingId] = useState(null);

  const loadCategories = useCallback(async () => {
    try {
      setLoading(true);
      setError("");

      const response = await getCategories({
        keyword,
        page,
        size,
        sort: "categoryName,asc",
      });

      const pageData = response.data;

      setCategories(pageData.content ?? []);
      setTotalPages(pageData.totalPages ?? 0);
      setTotalElements(pageData.totalElements ?? 0);
    } catch (err) {
      setError(
        err.response?.data?.message ??
          err.message ??
          "Cannot load categories"
      );
    } finally {
```

```
    setLoading(false);
  }
}, [keyword, page, size]);

useEffect(() => {
  loadCategories();
}, [loadCategories]);

function handleSearch(e) {
  e.preventDefault();

  setPage(0);
  setKeyword(keywordInput.trim());
}

function handleClearSearch() {
  setKeywordInput("");
  setKeyword("");
  setPage(0);
}

async function handleDelete(category) {
  const confirmed = window.confirm(
    `Delete category "${category.categoryName}"?`
  );

  if (!confirmed) {
    return;
  }

  try {
    setDeletingId(category.categoryId);
    setError("");

    await deleteCategory(category.categoryId);

    await loadCategories();
  } catch (err) {
    if (err.response?.status === 409) {
      setError(
        err.response?.data?.message ??
          "Cannot delete this category because it is being used by one or more news articles."
      );
    } else if (err.response?.status === 404) {
      setError("Category no longer exists.");
    } else {
      setError(
        err.response?.data?.message ??
          "Delete category failed."
      );
    }
  } finally {
    setDeletingId(null);
  }
}

return (
  <Container>
    <div className="d-flex justify-content-between align-items-center mb-3">
      <div>
        <h2 className="mb-1">
          Category Management
        </h2>

        <div className="text-muted">
          FUNewsManagementSystem
        </div>
      </div>

      <Button
        onClick={() => setShowCreate(true)}
      >
        Create Category
      </Button>
    </div>

    <Form
      onSubmit={handleSearch}
      className="mb-3"
    >
      <Row className="g-2">
        <Col md={6}>
          <Form.Control
            value={keywordInput}
            onChange={(e) =>
              setKeywordInput(e.target.value)
            }
            placeholder="Search category name..."
```

```
      />
    </Col>

    <Col xs="auto">
      <Button type="submit">
        Search
      </Button>
    </Col>

    <Col xs="auto">
      <Button
        type="button"
        variant="secondary"
        onClick={handleClearSearch}
      >
        Clear
      </Button>
    </Col>
  </Row>
</Form>

{error && (
  <Alert variant="danger">
    {error}
  </Alert>
)}

{loading ? (
  <div className="py-5 text-center">
    <Spinner animation="border" />
    <div className="mt-2">
      Loading categories...
    </div>
  </div>
) : (
  <>
    <div className="mb-2 text-muted">
      Total records: {totalElements}
    </div>

    <Table
      striped
      bordered
      hover
      responsive
      className="align-middle"
    >
      <thead>
        <tr>
          <th>ID</th>
          <th>Category Name</th>
          <th>Description</th>
          <th>Parent ID</th>
          <th>Status</th>
          <th style={{ width: 170 }}>
            Actions
          </th>
        </tr>
      </thead>

      <tbody>
        {categories.length === 0 ? (
          <tr>
            <td
              colSpan={6}
              className="text-center text-muted py-4"
            >
              No categories found.
            </td>
          </tr>
        ) : (
          categories.map((category) => (
            <tr key={category.categoryId}>
              <td>
                {category.categoryId}
              </td>

              <td>
                {category.categoryName}
              </td>

              <td>
                {category.categoryDescription}
              </td>

              <td>
                {category.parentCategoryId ??
                  "-"}
```

```
                    </td>

                    <td>
                      {category.active ? (
                        <Badge bg="success">
                          Active
                        </Badge>
                      ) : (
                        <Badge bg="secondary">
                          Inactive
                        </Badge>
                      )}
                    </td>

                    <td>
                      <Button
                        as={Link}
                        to={`/categories/${category.categoryId}/edit`}
                        size="sm"
                        className="me-2"
                      >
                        Edit
                      </Button>

                      <Button
                        variant="danger"
                        size="sm"
                        disabled={
                          deletingId ===
                          category.categoryId
                        }
                        onClick={() =>
                          handleDelete(category)
                        }
                      >
                        {deletingId ===
                        category.categoryId
                          ? "Deleting..."
                          : "Delete"}
                      </Button>
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </Table>

          <PaginationBar
            page={page}
            totalPages={totalPages}
            onPageChange={setPage}
          />
        </>
      )}

      <CategoryForm
        show={showCreate}
        onHide={() => setShowCreate(false)}
        onCreated={async () => {
          setPage(0);
          await loadCategories();
        }}
      />
    </Container>
  );
}
```

6.11. Bước 11 – Tạo Pagination Component

Tạo:

```
src/components/PaginationBar.jsx
import {
  Button,
  ButtonGroup,
} from "react-bootstrap";

export default function PaginationBar({
  page,
  totalPages,
  onPageChange,
}) {
  if (totalPages <= 1) {
    return null;
  }

  return (
    <div className="d-flex justify-content-between align-items-center mt-3">
      <div className="text-muted">
```

```
        Page {page + 1} / {totalPages}
      </div>

      <ButtonGroup>
        <Button
          variant="outline-primary"
          disabled={page <= 0}
          onClick={() =>
            onPageChange(page - 1)
          }
        >
          Previous
        </Button>

        <Button
          variant="outline-primary"
          disabled={page >= totalPages - 1}
          onClick={() =>
            onPageChange(page + 1)
          }
        >
          Next
        </Button>
      </ButtonGroup>
    </div>
  );
}
```

Verification

Browser DevTools → Network:

```
GET /api/categories?page=0&size=10&sort=categoryName,asc
```

Chuyển page:

```
GET /api/categories?page=1&size=10&sort=categoryName,asc
```

Search:

```
GET /api/categories?keyword=academic&page=0&size=10&sort=categoryName,asc
```

6.12. Bước 12 – CREATE: Reusable Category Form + Modal

Tạo:

```
src/components/CategoryForm.jsx
```

```
import { useState } from "react";

import {
  Alert,
  Button,
  Form,
  Modal,
} from "react-bootstrap";

import {
  createCategory,
} from "../api/categoryApi";

const initialForm = {
  categoryName: "",
  categoryDescription: "",
  parentCategoryId: "",
  active: true,
};

export default function CategoryForm({
  show,
  onHide,
  onCreated,
}) {
  const [form, setForm] =
    useState(initialForm);

  const [error, setError] =
    useState("");

  const [submitting, setSubmitting] =
    useState(false);

  function handleChange(e) {
    const {
      name,
      value,
      type,
      checked,
    } = e.target;

    setForm((current) => ({
      ...current,
      [name]:
        type === "checkbox"
          ? checked
          : value,
```

```
  }));
}

function validate() {
  if (!form.categoryName.trim()) {
    return "Category name is required.";
  }

  if (
    form.categoryName.trim().length > 100
  ) {
    return "Category name must not exceed 100 characters.";
  }

  if (
    !form.categoryDescription.trim()
  ) {
    return "Description is required.";
  }

  if (
    form.categoryDescription.trim()
      .length > 250
  ) {
    return "Description must not exceed 250 characters.";
  }

  return "";
}

async function handleSubmit(e) {
  e.preventDefault();

  const validationMessage =
    validate();

  if (validationMessage) {
    setError(validationMessage);
    return;
  }

  const payload = {
    categoryName:
      form.categoryName.trim(),

    categoryDescription:
      form.categoryDescription.trim(),

    parentCategoryId:
      form.parentCategoryId === ""
        ? null
        : Number(
            form.parentCategoryId
          ),

    active: form.active,
  };

  try {
    setSubmitting(true);
    setError("");

    await createCategory(payload);

    setForm(initialForm);

    onHide();

    await onCreated();
  } catch (err) {
    if (err.response?.status === 400) {
      setError(
        err.response?.data?.message ??
          "Invalid category data."
      );
    } else if (
      err.response?.status === 409
    ) {
      setError(
        err.response?.data?.message ??
          "Category name already exists."
      );
    } else {
      setError(
        err.response?.data?.message ??
          "Create category failed."
      );
    }
```

```
  } finally {
    setSubmitting(false);
  }
}

function handleClose() {
  if (submitting) {
    return;
  }

  setError("");
  setForm(initialForm);
  onHide();
}

return (
  <Modal
    show={show}
    onHide={handleClose}
    centered
  >
    <Form onSubmit={handleSubmit}>
      <Modal.Header closeButton>
        <Modal.Title>
          Create Category
        </Modal.Title>
      </Modal.Header>

      <Modal.Body>
        {error && (
          <Alert variant="danger">
            {error}
          </Alert>
        )}

        <Form.Group className="mb-3">
          <Form.Label>
            Category Name
          </Form.Label>

          <Form.Control
            name="categoryName"
            value={form.categoryName}
            onChange={handleChange}
            maxLength={100}
            required
          />
        </Form.Group>

        <Form.Group className="mb-3">
          <Form.Label>
            Description
          </Form.Label>

          <Form.Control
            as="textarea"
            rows={3}
            name="categoryDescription"
            value={
              form.categoryDescription
            }
            onChange={handleChange}
            maxLength={250}
            required
          />
        </Form.Group>

        <Form.Group className="mb-3">
          <Form.Label>
            Parent Category ID
          </Form.Label>

          <Form.Control
            type="number"
            min="1"
            name="parentCategoryId"
            value={
              form.parentCategoryId
            }
            onChange={handleChange}
            placeholder="Optional"
          />

          <Form.Text muted>
            Leave empty if this is a
            root category.
          </Form.Text>
        </Form.Group>
```

```
          <Form.Check
            type="switch"
            name="active"
            label="Active"
            checked={form.active}
            onChange={handleChange}
          />
        </Modal.Body>

        <Modal.Footer>
          <Button
            type="button"
            variant="secondary"
            disabled={submitting}
            onClick={handleClose}
          >
            Cancel
          </Button>

          <Button
            type="submit"
            disabled={submitting}
          >
            {submitting
              ? "Saving..."
              : "Create"}
          </Button>
        </Modal.Footer>
      </Form>
    </Modal>
  );
}
```

Verification

Submit:

```
{
  "categoryName": "Technology News",
  "categoryDescription": "Technology and software engineering news",
  "parentCategoryId": null,
  "active": true
}
```

Expected:

```
POST /api/categories
HTTP 201 Created
```

Sau đó:

1. Modal đóng.
2. Category list refresh.
3. Record mới xuất hiện.
4. SQL Server có row mới.

Query:

```
SELECT TOP 10 *
FROM dbo.Category
ORDER BY CategoryID DESC;
```

PASS chỉ khi UI + HTTP + DB đều đúng.

6.13. Bước 13 – UPDATE: `EditCategory.jsx`

Tạo:

```
src/pages/EditCategory.jsx
import {
  useEffect,
  useState,
} from "react";

import {
  Alert,
  Button,
  Container,
  Form,
  Spinner,
} from "react-bootstrap";

import {
  useNavigate,
  useParams,
} from "react-router-dom";

import {
  getCategoryById,
  updateCategory,
} from "../api/categoryApi";

export default function EditCategory() {
  const { id } = useParams();
  const navigate = useNavigate();
```

```
const [form, setForm] =
  useState(null);

const [loading, setLoading] =
  useState(true);

const [submitting, setSubmitting] =
  useState(false);

const [error, setError] =
  useState("");

useEffect(() => {
  let ignore = false;

  async function loadCategory() {
    try {
      setLoading(true);
      setError("");

      const response =
        await getCategoryById(id);

      if (!ignore) {
        const category =
          response.data;

        setForm({
          categoryName:
            category.categoryName ??
            "",

          categoryDescription:
            category.categoryDescription ??
            "",

          parentCategoryId:
            category.parentCategoryId ??
            "",

          active:
            category.active ?? true,
        });
      }
    } catch (err) {
      if (!ignore) {
        if (
          err.response?.status === 404
        ) {
          setError(
            `Category ${id} not found.`
          );
        } else {
          setError(
            err.response?.data
              ?.message ??
              "Cannot load category."
          );
        }
      }
    } finally {
      if (!ignore) {
        setLoading(false);
      }
    }
  }

  loadCategory();

  return () => {
    ignore = true;
  };
}, [id]);

function handleChange(e) {
  const {
    name,
    value,
    type,
    checked,
  } = e.target;

  setForm((current) => ({
    ...current,
    [name]:
      type === "checkbox"
        ? checked
```

```
        : value,
  }));
}

async function handleSubmit(e) {
  e.preventDefault();

  if (!form.categoryName.trim()) {
    setError(
      "Category name is required."
    );
    return;
  }

  if (
    !form.categoryDescription.trim()
  ) {
    setError(
      "Description is required."
    );
    return;
  }

  const payload = {
    categoryName:
      form.categoryName.trim(),

    categoryDescription:
      form.categoryDescription.trim(),

    parentCategoryId:
      form.parentCategoryId === ""
        ? null
        : Number(
            form.parentCategoryId
          ),

    active: form.active,
  };

  try {
    setSubmitting(true);
    setError("");

    await updateCategory(
      id,
      payload
    );

    navigate("/categories");
  } catch (err) {
    if (err.response?.status === 400) {
      setError(
        err.response?.data?.message ??
          "Invalid category data."
      );
    } else if (
      err.response?.status === 404
    ) {
      setError(
        `Category ${id} not found.`
      );
    } else if (
      err.response?.status === 409
    ) {
      setError(
        err.response?.data?.message ??
          "Category update conflicts with existing data."
      );
    } else {
      setError(
        err.response?.data?.message ??
          "Update category failed."
      );
    }
  } finally {
    setSubmitting(false);
  }
}

if (loading) {
  return (
    <Container className="py-5 text-center">
      <Spinner animation="border" />
      <div className="mt-2">
        Loading category...
      </div>
    </Container>
```

```
  );
}

return (
  <Container style={{ maxWidth: 720 }}>
    <h2>
      Edit Category #{id}
    </h2>

    <div className="text-muted mb-3">
      FUNewsManagementSystem
    </div>

    {error && (
      <Alert variant="danger">
        {error}
      </Alert>
    )}

    {form && (
      <Form onSubmit={handleSubmit}>
        <Form.Group className="mb-3">
          <Form.Label>
            Category Name
          </Form.Label>

          <Form.Control
            name="categoryName"
            value={form.categoryName}
            onChange={handleChange}
            maxLength={100}
            required
          />
        </Form.Group>

        <Form.Group className="mb-3">
          <Form.Label>
            Description
          </Form.Label>

          <Form.Control
            as="textarea"
            rows={4}
            name="categoryDescription"
            value={
              form.categoryDescription
            }
            onChange={handleChange}
            maxLength={250}
            required
          />
        </Form.Group>

        <Form.Group className="mb-3">
          <Form.Label>
            Parent Category ID
          </Form.Label>

          <Form.Control
            type="number"
            min="1"
            name="parentCategoryId"
            value={
              form.parentCategoryId
            }
            onChange={handleChange}
          />
        </Form.Group>

        <Form.Check
          type="switch"
          name="active"
          label="Active"
          checked={form.active}
          onChange={handleChange}
          className="mb-4"
        />

        <Button
          type="submit"
          disabled={submitting}
          className="me-2"
        >
          {submitting
            ? "Saving..."
            : "Save Changes"}
        </Button>
```

```
          <Button
            type="button"
            variant="secondary"
            disabled={submitting}
            onClick={() =>
              navigate("/categories")
            }
          >
            Cancel
          </Button>
        </Form>
      )}
    </Container>
  );
}
```

Verification

Mở:

```
/categories/1/edit
```

Execution:

```
GET /api/categories/1
        ↓
form prefill
        ↓
user edits values
        ↓
PUT /api/categories/1
        ↓
200 OK
        ↓
navigate("/categories")
```

Kiểm DB:

```
SELECT *
FROM dbo.Category
WHERE CategoryID = 1;
```

6.14. Bước 14 – DELETE và Business Rule từ Slot 22

Đây là điểm quan trọng giúp **frontend Slot 23 đồng bộ đúng business logic backend Slot 22**.
Backend không cho xóa Category đang được NewsArticle tham chiếu.
Frontend **không được giả định mọi DELETE đều thành công**.

Case A – Category không được sử dụng

```
DELETE /api/categories/{id}
```

Expected:

```
204 No Content
```

Frontend:

- row biến mất;
- refresh list;
- GET id sau đó → `404`;
- SQL row không còn.

Case B – Category đang được NewsArticle sử dụng

Expected:

```
409 Conflict
```

Frontend phải hiển thị:

```
Cannot delete this category because it is being used
by one or more news articles.
```

Không được:

- xóa row khỏi UI trước response;
- coi 409 là network error chung;
- reload page bằng `window.location.reload()`.

6.15. Bước 15 – Negative Test Cases

ID	Test	Cách tạo	Expected
N01	Backend stopped	Stop Spring Boot	Network error, UI không crash
N02	Wrong API URL	Sai `VITE_API_URL`	Connection error
N03	Empty name	Submit blank	Client validation, không POST
N04	Empty description	Submit blank	Client validation
N05	Backend validation	Payload invalid	`400` + error UI
N06	Duplicate name	Create existing name	`409`
N07	Missing edit ID	`/categories/32767/edit`	`404` state
N08	Referenced delete	Delete used Category	`409`, row vẫn còn
N09	CORS failure	Remove frontend allowed origin	Browser CORS error
N10	Double delete	Click nhanh nhiều lần	button disabled/pending
N11	Backend Page mistaken as Array	`setCategories(response.data)`	render/runtime error
N12	Wrong property	`categoryDescription` mismatch	field/payload issue

6.16. Bước 16 – Trace End-to-End bằng Browser DevTools

A. Trace READ

Mở:

```
DevTools → Network → Fetch/XHR
```

Reload Category page.
Quan sát:
`GET /api/categories?page=0&size=10&sort=categoryName,asc`
Trace:

```
CategoryList useEffect
    ↓
loadCategories()
    ↓
getCategories()
    ↓
axiosClient.get()
    ↓
Spring CategoryController.getAll()
    ↓
CategoryService.findAll()
    ↓
CategoryRepository
    ↓
SQL Server SELECT
    ↑
Page<CategoryResponse>
    ↑
Axios response
    ↓
response.data.content
    ↓
setCategories()
    ↓
React render Table
```

B. Trace CREATE

```
CategoryForm.handleSubmit
    ↓
client validation
    ↓
createCategory(payload)
    ↓
POST /api/categories
    ↓
CategoryController.create()
    ↓
CategoryServiceImpl.create()
    ↓
duplicate rule
    ↓
CategoryRepository.save()
    ↓
Hibernate INSERT
    ↓
SQL Server COMMIT
    ↑
201 CategoryResponse
    ↑
onCreated()
    ↓
loadCategories()
    ↓
React re-render
```

Evidence bắt buộc

1. Browser Network request.
2. Request payload.
3. HTTP response.
4. Backend debugger/log.
5. SQL query/result.
6. UI before/after.

6.17. Bước 17 – Break-It → Diagnose → Fix Workshop

Break	Triệu chứng	Diagnose	Fix
Dùng `response.data` như array	`.map is not a function`	Inspect Page JSON	dùng `response.data.content`
Bỏ dependency/useCallback logic sai	repeated requests	Network	sửa effect dependencies
Hard-code URL trong page	môi trường khó đổi	Code review	API client + `.env`
`.env` đổi nhưng không restart	vẫn URL cũ	Console/Network	restart Vite
Gửi `parentCategoryId: ""`	backend bind/validation bất ổn	Request payload	convert `""` → `null`
Gửi string `"1"` thay number	contract không rõ	payload inspect	`Number(...)`
Không catch 409	generic error	inspect status	branch `status === 409`
DELETE success nhưng không refresh	stale row	UI state	refetch/update state
Search nhưng không reset page	page rỗng	state trace	`setPage(0)`
Update thiếu selected ID	wrong URL	Network	`useParams()`
Sai `categoryDescription`	400 hoặc field mất	request payload	match DTO contract
Dùng DB column `CategoryDesciption` ở React	contract coupling	DTO review	frontend dùng `categoryDescription`

Điểm cuối rất quan trọng: frontend phải tuân theo **REST DTO contract**, không phụ thuộc trực tiếp vào tên cột SQL Server. Backend Slot 22 cố ý che chi tiết schema `CategoryDesciption` thông qua `CategoryResponse.categoryDescription`.

6.18. Bài tập chủ đề

Bài	Yêu cầu	Evidence
1. READ	GET Category Page	Network + Table
2. SEARCH	keyword query	Request URL + filtered content
3. PAGINATION	Previous/Next	page metadata
4. CREATE	Modal + POST + 201	payload + DB
5. UPDATE	GET detail + PUT	before/after
6. DELETE	DELETE free Category	204 + DB
7. BUSINESS DELETE	referenced Category	409 + row retained
8. VALIDATION	required/max length	negative evidence
9. ERROR UX	400/404/409	UI Alert
10. API MODULE	no page-level hard-coded HTTP	code review
11. TRACE	READ + CREATE lifecycle	sequence note
12. BREAK-IT	tạo và sửa ít nhất 3 lỗi	Failure → diagnosis → fix

6.19. Bài tập tổng hợp hệ thống – FUNews Category React CRUD Client

Sinh viên phải hoàn thành một SPA quản lý Category có:

1. Category list.
2. Search theo category name.
3. Pagination.
4. Create Category.
5. Edit Category.
6. Delete Category.
7. Hiển thị Active/Inactive.
8. Parent Category ID.
9. Client validation.
10. Backend validation handling.
11. 404 handling.
12. 409 duplicate handling.
13. 409 referenced-category delete handling.
14. Loading state.
15. Empty state.
16. Mutation pending state.
17. Error state.
18. API module tách riêng.
19. Routing.
20. Không hard-code backend URL trong component.

6.20. Integration Gate – Frontend ↔ Slot 22 Backend

Gate	Yêu cầu	Evidence
G1	Vite build/run	`npm run dev`
G2	API config	`.env`
G3	Backend contract	GET `/api/categories`
G4	Page parsing	`response.data.content`
G5	READ	list đúng DB
G6	SEARCH	keyword hoạt động
G7	PAGINATION	page change đúng
G8	CREATE	POST `201`
G9	UPDATE	PUT `200`
G10	DELETE	DELETE `204`
G11	VALIDATION	`400` handled
G12	NOT FOUND	`404` handled
G13	CONFLICT	`409` handled
G14	BUSINESS RULE	referenced Category không bị xóa
G15	STATE	loading/error/empty/pending
G16	ARCHITECTURE	Page → API module → backend
G17	TRACE	Browser → Controller → Service → Repository → SQL
G18	EXPLAIN	Sinh viên giải thích được full-stack lifecycle

6.21. Test Matrix bắt buộc

ID	Frontend action	Backend expected	UI expected
T01	Load Category page	GET `200`	table render
T02	Search academic	GET `200`	filtered rows
T03	Next page	GET `200`	new page
T04	Open existing edit	GET `200`	prefilled form
T05	Open missing edit	GET `404`	error state
T06	Create valid	POST `201`	new row
T07	Create invalid	POST not sent / `400`	validation
T08	Duplicate category	POST `409`	conflict message
T09	Update existing	PUT `200`	updated row
T10	Update missing	PUT `404`	error state
T11	Delete free category	DELETE `204`	row removed
T12	Delete referenced category	DELETE `409`	row retained
T13	Backend stopped	network failure	app survives
T14	CORS blocked	browser CORS	useful error/debug evidence
T15	Double mutation	one effective action	pending button

6.22. Bài tập mở rộng – Chuyển từ Category sang FUNewsManagementSystem hoàn chỉnh

Sau khi Category Client PASS, sinh viên thiết kế tiếp frontend cho:

```
FUNewsManagementSystem
├─ Category Management
├─ News Article Management
├─ System Account Management
└─ Authentication / Authorization
```

A. NewsArticle client

Dự kiến:

```
pages/
├─ NewsArticleList.jsx
├─ EditNewsArticle.jsx

components/
└─ NewsArticleForm.jsx

api/
└─ newsArticleApi.js
```

Các field phải lấy từ **Assignment 02/backend contract**, không tự suy đoán.

B. Category selector trong NewsArticleForm

Thay vì nhập:

```
categoryId = 3
```

bằng tay, frontend phải:

```
GET /api/categories
        ↓
render <select>
        ↓
user chọn Category
        ↓
send categoryId
```

C. Business-rule-aware UI

Nếu:

```
DELETE Category
→ 409 CONFLICT
```

Frontend cần giải thích:

```
Category cannot be deleted because it is referenced
by one or more news articles.
```

Không nên chỉ hiển thị:

```
Error 409
```

6.23. Refactor nâng cao

Sau khi phiên bản baseline hoạt động, sinh viên có thể tự nghiên cứu:

R1. Custom Hook

```
useCategories()
```

Tách:

```
loading
error
categories
pagination
loadCategories
```

khỏi page component.

R2. Reusable Category Form

Dùng cùng một form cho:

```
CREATE
UPDATE
```

thay vì duplicate field definitions.

R3. Centralized Axios error normalizer

Ví dụ:

```
export function getApiErrorMessage(
  error,
  fallback = "Request failed"
) {
  return (
    error.response?.data?.message ??
    error.message ??
    fallback
  );
}
```

R4. Request/Response Mapper

Không truyền raw backend response xuyên toàn application.

Ví dụ:

```
export function mapCategoryResponse(data) {
  return {
```

```
    id: data.categoryId,
    name: data.categoryName,
    description:
      data.categoryDescription,
    parentId:
      data.parentCategoryId,
    active: data.active,
  };
}
```

R5. Abort request

Sử dụng:
`AbortController`
để tránh stale response khi component unmount/search thay đổi nhanh.

R6. Debounced Search

Thay Search button bằng:
`input → delay 300–500 ms → GET search`
Nhưng phải tránh gửi request trên từng keystroke không kiểm soát.

R7. URL Query Parameters

Đồng bộ:
`/categories?keyword=academic&page=2`
với React state để:

- refresh không mất search;
- back/forward browser đúng;
- URL shareable.

R8. React Query / TanStack Query – Research Track

Sinh viên so sánh:
`manual useEffect/useState`
`vs`
`TanStack Query`
về:

- server state;
- cache;
- refetch;
- mutation;
- stale state;
- loading/error management.

Đây là **advanced research**, không thay baseline nếu course chưa yêu cầu library này.

KẾT LUẬN FULL-STACK CỦA SLOT 22 → SLOT 23

```
SLOT 22 – BACKEND

React Request Contract
        ↑
@RestController
        ↓
Service
        ↓
Repository
        ↓
Spring Data JPA
        ↓
Hibernate
        ↓
SQL Server


SLOT 23 – FRONTEND

User
 ↓
React Component/Page
 ↓
Event Handler
 ↓
API Module
 ↓
Axios
 ↓
HTTP REST
 ↓
SLOT 22 Backend
 ↓
HTTP Response
 ↓
React State
 ↓
Re-render UI
```

Sinh viên phải giải thích được đầy đủ:

```
User action
→ React event
→ state/form
→ Axios request
→ REST endpoint
→ Controller
→ Service
→ Repository
→ Hibernate
→ SQL Server
→ HTTP response
→ React state
→ UI render
```

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa ngắn	Ví dụ Slot 23
Component	Đơn vị UI dạng function/class; course dùng function component	ListOfOrchids
JSX	Cú pháp mô tả UI trong JavaScript	<Table>...</Table>
Props	Input từ parent vào component	show, onHide, onCreated
State	Dữ liệu thay đổi làm UI re-render	orchids, form, loading
useState	Hook quản lý state cục bộ	const [error,setError] = useState("")
useEffect	Hook đồng bộ với external system sau commit	GET list
Controlled input	Input value/checked do state điều khiển	value={form.name}
Event handler	Hàm phản ứng user event	handleSubmit
Axios instance	HTTP client cấu hình dùng chung	axiosClient
baseURL	URL prefix cho request	VITE_API_URL
Promise	Đại diện kết quả async	axios.get(...)
async/await	Cú pháp xử lý Promise	await createOrchid(form)
Mutation	Request làm thay đổi server state	POST/PUT/DELETE
Refetch	Gọi lại GET để đồng bộ server state	loadOrchids()
Optimistic UI	UI cập nhật trước server success	production extension
Pessimistic UI	Chờ server success rồi cập nhật	course-friendly
CORS	Browser policy cho cross-origin request	5173 → 8080
Preflight	OPTIONS kiểm tra CORS trước một số request	browser-generated
Environment variable	Cấu hình theo môi trường build/run	VITE_API_URL
React Router	Client-side routing	/orchids/:id/edit
Route parameter	Biến trong URL route	id
Form validation	Kiểm tra input trước submit	required name
HTTP status	Kết quả protocol	200,201,204,400,404
Network tab	DevTools theo dõi HTTP	Fetch/XHR
Interceptor	Hook trước request/sau response của Axios	token/error normalization
AbortController	API hủy async request hỗ trợ signal	request cancellation
Server state	Dữ liệu nguồn ở backend	orchids list
Client/UI state	Dữ liệu cục bộ cho giao diện	modal open
Stale state	UI/state không còn phản ánh server hiện tại	sau mutation không refresh
Error boundary	Boundary bắt render errors trong React	production extension

VIII. RESEARCH BASE VÀ AI BASE LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

Mọi prompt phải được dùng theo nguyên tắc: AI output = hypothesis/learning aid. Sinh viên phải kiểm chứng bằng source, Network/Console, backend log và official documentation. Với claim “current/latest”, bắt buộc ghi version/date/source chính thức.

VIII.1. Nhóm I – Phân tích workflow và quy luật thực thi

Prompt I.1

Phân tích startup/render flow của React project từ main.jsx → BrowserRouter → App → route → ListOfOrchids. Với mỗi bước ghi file, function, state/props và evidence tôi có thể đặt console/breakpoint.

Prompt I.2

Trace một GET /orchids/ từ useEffect đến Axios, browser Network, Spring Boot và quay về setOrchids/re-render. Phân biệt code của tôi với framework/browser internals.

Prompt I.3

Trace một POST create từ controlled form onChange → form state → onSubmit → Axios → response → refetch/update state → UI. Chỉ dùng method thực sự có trong source tôi cung cấp.

Prompt I.4

Giải thích vì sao useEffect có thể chạy lại và cách dependency array quyết định synchronization. Tạo 5 lỗi điển hình và experiment để tôi tự chứng minh.

Prompt I.5

Phân tích render snapshot và stale closure trong một CRUD component; đưa ví dụ bug có thể xảy ra với async request và state.

Prompt I.6

Trace DELETE workflow và giải thích vì sao disable button theo deletingId giúp UX nhưng không thay server-side integrity.

Prompt I.7

Phân tích route-based edit: URL → useParams → GET detail → prefill controlled form → PUT → navigate. Đề xuất breakpoint/Network evidence.

Prompt I.8

Giải thích browser CORS/preflight trong request PUT/DELETE của project, phân biệt lỗi CORS với backend 4xx/5xx.

Prompt I.9

Vẽ state machine cho Create Modal gồm CLOSED, EDITING, SUBMITTING, SUCCESS, ERROR và map state này vào React state variables.

Prompt I.10

Tạo checklist 15 câu để sinh viên tự trace source bằng debugger/Network thay vì trả lời thuộc lòng.

VIII.2. Nhóm II – Kỹ thuật nâng cao và hệ sinh thái frontend

Prompt II.1

Phân tích 3 chiến lược sau mutation: refetch, patch local state, optimistic update. Dùng Orchid CRUD làm ví dụ; so sánh consistency, complexity, request count và rollback.

Prompt II.2

Giải thích race condition khi user đổi route/id nhanh trong Edit page. So sánh ignore flag, AbortController và server-state library cancellation.

Prompt II.3

Phân tích khi nào nên tách custom hook useOrchids/useOrchidForm. Đưa tiêu chí “abstraction pressure” để tránh over-engineering.

Prompt II.4

So sánh controlled vs uncontrolled forms và FormData trong React hiện đại. Với CRUD admin form 6 fields, chọn approach và giải thích trade-off.

Prompt II.5

Thiết kế reusable OrchidForm dùng chung Create/Edit: props, initialValues, validation, submit callback, reset lifecycle; tránh state bị cũ khi đổi selected item.

Prompt II.6

Phân tích Axios interceptors: token, correlation ID, error normalization, refresh token. Chỉ ra 5 anti-pattern khi dùng global interceptor.

Prompt II.7

Thiết kế debounce search + cancellation cho GET /orchids?name=. Giải thích stale response và cách kiểm chứng bằng Network timing.

Prompt II.8

So sánh useState/useEffect baseline với TanStack Query/SWR-like server-state abstraction. Chỉ ra pain point nào đủ để chuyển công cụ.

Prompt II.9

Phân tích error boundary vs async API error handling. Tạo ma trận loại lỗi → boundary xử lý phù hợp.

Prompt II.10

Thiết kế testing strategy cho React CRUD client: unit validation, component interaction, mocked network, integration với backend, E2E. Đề xuất evidence cho từng layer.

VIII.3. Nhóm III – Kỹ thuật chuyên nghiệp trong dự án production

Prompt III.1

Đóng vai Senior Frontend Engineer review CRUD client hiện tại. Đánh giá module boundaries, API layer, form state, error UX, accessibility, testability và đưa refactor roadmap theo mức ưu tiên.

Prompt III.2

Thiết kế API client production cho nhiều resource Product/Category/Order: baseURL, timeout, headers, interceptors, error model, cancellation và dependency injection/testing.

Prompt III.3

Phân tích strategy quản lý configuration dev/staging/prod với Vite. Nêu rõ cái gì public, cái gì tuyệt đối không nằm trong client bundle, và deployment implications.

Prompt III.4

Thiết kế standardized frontend error model map từ backend ProblemDetail/error DTO sang field errors, toast/page errors và retry behavior.

Prompt III.5

Phân tích accessibility cho CRUD form/modal: label, focus trap, error focus, keyboard, aria-live, destructive confirmation. Tạo review checklist.

Prompt III.6

Thiết kế concurrency UX khi backend dùng optimistic locking/version field: edit stale item, 409 conflict, reload/merge/retry options.

Prompt III.7

Phân tích security boundary của SPA: XSS, token storage, CSRF, CORS, authorization. Chỉ ra việc nào frontend làm được và việc nào bắt buộc backend enforce.

Prompt III.8

Thiết kế observability cho frontend: structured client logs, error telemetry, correlation/trace ID nối với backend, privacy considerations.

Prompt III.9

So sánh modal CRUD admin vs route-based CRUD cho hệ thống lớn: deep-link, history, accessibility, code splitting, workflow complexity, mobile responsiveness.

Prompt III.10

Tạo PR review checklist 25 mục cho React CRUD feature trước merge: contract, state, effect, form, error, accessibility, performance, security, tests, environment.

VIII.4. Nhóm IV – Xu hướng cập nhật kiến thức và cách cộng đồng hiện tại sử dụng

Prompt IV.1

Kiểm tra official React documentation hiện tại và tổng hợp khuyến nghị mới nhất liên quan forms, Effects và data fetching. Ghi version/date/source; phân biệt kiến thức course baseline với current direction.

Prompt IV.2

Kiểm tra official Vite docs hiện tại về env/modes/proxy/build. Tạo bảng MUST KNOW NOW / SHOULD EXPLORE / WATCH LATER cho sinh viên SBA301.

Prompt IV.3

Kiểm tra Axios documentation/changelog chính thức về cancellation, interceptors, adapters và error handling. Chỉ ra API nào nên/không nên dùng trong project mới.

Prompt IV.4

Khảo sát current community pattern cho server state (TanStack Query, SWR hoặc tương đương) nhưng phải đối chiếu official docs; giải thích vì sao useEffect vẫn cần học ở course core.

Prompt IV.5

Khảo sát current React form ecosystem (React Hook Form/schema validation approaches). Chỉ đề xuất khi có pain point; so sánh với controlled form thuần.

Prompt IV.6

Kiểm tra React Router current official guidance về data routing/navigation/forms nếu có thay đổi; so sánh với BrowserRouter + Routes baseline của course.

Prompt IV.7

Tổng hợp xu hướng testing React hiện tại từ official/tool docs: Testing Library, Vitest, Playwright/Cypress-like E2E. Tạo lộ trình nâng cấp từ manual test.

Prompt IV.8

Khảo sát frontend observability hiện tại: Web Vitals, error tracking, distributed tracing context. Phân loại kiến thức nào phù hợp sinh viên năm hiện tại.

Prompt IV.9

Phân tích xu hướng BFF/API gateway/reverse proxy cho SPA deployment và tác động đến API URL/CORS so với localhost dev architecture.

Prompt IV.10

Tạo “React CRUD Client Technology Radar 2026”: React core, Vite, Axios/fetch, Router, forms, server-state, testing, accessibility, observability; mỗi mục ghi official source, maturity và recommendation.

VIII.5. AI Verification Worksheet

Prompt/claim AI	Nguồn kiểm chứng	Thử nghiệm	Kết luận SV
useEffect dependency behavior	React official docs + source	console/Network count	
Axios error taxonomy	Axios docs + observed error object	stop backend / 400	
VITE_* exposure	Vite official docs + built bundle	console/build inspect	
Mutation strategy	source + Network + DB	refetch vs local patch	
Current ecosystem claim	official release/docs + access date	small spike/POC	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

9.1. Self-check – kiến thức

- Tôi giải thích được render → commit → Effect thay vì chỉ nói “useEffect chạy khi load page”.

- Tôi phân biệt server state, form state, UI state và async state.
- Tôi viết được Axios instance và biết tại sao không hard-code URL rải rác.
- Tôi giải thích controlled input, checkbox checked và type conversion.
- Tôi map được CRUD UI → HTTP method → expected status.
- Tôi biết client validation không thay backend validation.
- Tôi phân biệt CORS error, network error, HTTP 4xx/5xx và render error.
- Tôi giải thích refetch vs local update sau mutation.
- Tôi trace được GET và POST bằng Network tab.
- Tôi biết VITE_* là public client configuration, không phải nơi lưu secret.

9.2. Self-check – thực hành

Checkpoint	PASS khi
Project start	npm run dev chạy, không lỗi build
Config	API base URL đọc từ env
READ	List render dữ liệu backend
CREATE	POST success + item mới xuất hiện + DB verify
UPDATE	Form prefill + PUT + dữ liệu đổi
DELETE	Confirm + request success + row biến mất + DB verify
Forms	Boolean/text fields map đúng payload
Error	Ít nhất 2 negative case hiển thị đúng
Trace	Có ảnh Network + method chain
Explain	Có thể giải thích code không nhìn mẫu

9.3. 15 câu oral check gợi ý

1. Tại sao fetch API trong thân component là sai?
2. useEffect([]) khác useEffect() như thế nào về hành vi?
3. Tại sao checkbox phải dùng checked thay vì value?
4. Vì sao VITE_API_URL không phải secret?
5. error.response và error.request khác nhau thế nào trong Axios?
6. Sau POST, vì sao nên dùng server response hoặc refetch?
7. Khi DELETE trả 204, frontend đọc response body có nên kỳ vọng JSON không?
8. Tại sao Postman chạy nhưng browser fail có thể là CORS?
9. Khi backend trả 400 field validation, UI nên xử lý thế nào?
10. Khi route id thay đổi nhanh, race condition có thể xảy ra ở đâu?
11. Khi nào modal edit tốt hơn route edit? Khi nào ngược lại?
12. Tại sao server-state library không thay kiến thức HTTP/state nền tảng?
13. UI ẩn nút Delete có phải authorization không?
14. Làm sao chứng minh một mutation đã thành công ngoài việc nhìn UI?
15. Nếu request 200 nhưng UI vẫn cũ, bạn debug theo thứ tự nào?

9.4. Definition of Done – Slot 23

- CRUD client hoạt động end-to-end với backend thật.
- Code có separation tối thiểu giữa page/component và API module.
- Forms là controlled và xử lý boolean đúng.
- Có loading/error/pending state.
- Có Network evidence và DB hậu điều kiện cho mutation.
- Có ít nhất 2 Break-It/Fix notes.
- Hoàn thành AI Verification Worksheet cho ít nhất 3 prompt.
- Sinh viên giải thích được workflow bằng lời và source trace.

TÀI LIỆU THAM CHIẾU VÀ SOURCE MAPPING

Course baseline: Chapter 15 – Implement CRUD Application with RESTful Web Service and ReactJS, Part 2 (ReactJS project); Lab 05; Slot 20 full-stack SPA; Slot 22 CRUD backend; Assignment 02 FUNewsManagementSystem.

Official update references (production awareness):

- https://react.dev/reference/react/useEffect
- https://react.dev/reference/react-dom/components/input
- https://vite.dev/guide/env-and-mode
- https://axios-http.com/

NOTE: Course baseline và current ecosystem phải được phân biệt. Không ép nâng version giữa kỳ nếu làm ảnh hưởng reproducibility của lab; mọi claim latest/current phải được kiểm bằng official docs tại thời điểm sử dụng.